

© Brendan Choi 2021

B. Choi, *Introduction to Python Network Automation*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-6806-3_17

17. Upgrading Multiple Cisco IOS XE Routers

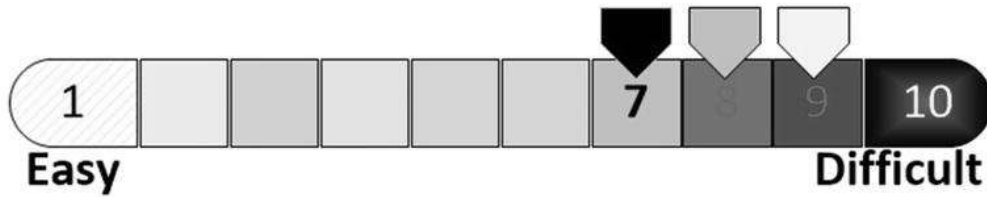
Brendan Choi¹

(1) Sydney, NSW, Australia

This chapter explains object-oriented programming (OOP) by using a real production example. Then the chapter will discuss Python application flow controls using a username and password application example. Finally, we will discuss the logical thinking processes involved in planning an IOS upgrade.

IOS upgrades on Cisco devices are a critical part of network operations to add reliability and security to all Cisco devices. Regular IOS patch management is also part of the network team's key performance indicators, so it is a critical part of the business. You will be guided through installing two Cisco CRS1000v IOS XE routers to prepare you for the next two chapters. Finally, we will discuss the manual Cisco IOS upgrade processes and tasks performed by a network engineer and translate the manual tasks and engineer's thinking into Python scripts. All tasks and processes must be documented in a process flowchart before writing the Python code. If you want to automate something, you must write it, documenting each step.

****Difficulty level will be relative to reader's previous experience
with Cisco Networking Technology.***



Practical Python Network Automation Lab: IOS XE Upgrade Lab

I hope your initial enthusiasm for your Python network automation journey has not worn off too much along the way. If you enjoyed every chapter leading up to this one, that is fantastic. However, you probably encountered a few obstacles and need to work on those areas more. Every chapter was designed to teach you some new IT skills to empower you. Get ready for the most exciting three chapters in this book!

Throughout this book, you have learned various IT system management skills, and finally you are here. Like many other popular programming languages, Python is classified as an object-oriented programming language. I have left the OOP discussion until now because a discussion of OOP could easily scare away novice Python learners. In this chapter, I will give a working example of how OOP can be applied to a real work scenario so you can relate OOP immediately to your work. Next, you will review how we can use the basic flow controls using the user ID and password inputs in our scripts. In many environments, you may not have a secure password vault; often you will write interactive code, requiring the users to provide the user ID and password to the application you have developed. In the last labs, you will build two Cisco CSR 1000v virtual routers on VMware Workstation 15 Pro to set you up for mini tools development in Chapter [18](#). We will discuss the Cisco IOS upgrade process in detail at the end of this chapter.

Applying OOP Concepts to Your Network

Until this point, we have not discussed object-oriented programming in Python. Even without knowing a lot about OOP, our Python scripts were working fine. But when intermediate to advanced Python users start talking about optimizing Python scripts, they always emphasize that Python is an OOP language. So, we have to take full advantage of the OOP concepts. Just like with another popular OOP language, Java, we need to know four fundamental concepts: encapsulation, abstraction, inheritance, and polymorphism. The ultimate aim of OOP is to bind the data better so a lot of duplication is removed and the same code can be reused throughout the same program or application.

Let's quickly cover the basics, and then we will look at OOP in action by writing an OOP example using routers and switches. We'll keep the theory to the minimum and get into the practical example to help you better understand the OOP concepts.

- *Object-oriented programming* : From an OOP perspective, everything is considered an object. The C language is called a *procedure-oriented programming* language because it runs based on the processes running in a functional order. On the other hand, in OOP, the objects are related to each other and connected to run the program. OOP treats anything and everything as an object, and it is viewed that they are linked to each other by some relationship.
- *Object*: As the word suggests, an *object* in OOP is a thing or an object. For example, a human or a robot can be considered an object, and also, a book or a router is an object. Since the same router (or switches) models look the same, every router can be called an object because if you make a dent on one of the routers, it does not necessarily make dents on other routers. Objects with similar characteristics or features are in the same group of objects, meaning that they can be grouped. Certain characteristics are the same or similar between the same objects in a common group. Even we humans can be considered as objects based on the Python OOP concept.
- *Class*: People generally have almost identical attributes such as two eyes, a nose, a mouth, hands, feet, and other physical body parts. Books also have the same attributes, such as book title, author, publisher, and publication dates. Switches from different vendors will still have the same switch model, serial number, rack unit size, and

Ethernet ports. Classes are defined by a collection of common properties of objects such as people, books, and switches have in common.

- *Abstraction*: This refers to showing only essential attributes and hide unnecessary objects from the user. For example, a complex function in a class hides (abstracts) detailed information from the user, so s/he can implement logic on top of abstraction without understanding the actual function or even thinking about all the hidden complexities of the class.
- *Encapsulation*: This refers to the idea of binding the data and methods which manipulate the data into a unit. Again, a class is an excellent example of encapsulation as the objects inside a class keep their state private and not directly accessible; instead, the state of the objects are accessed through methods by calling a list of public functions.
- *Polymorphism*: This is the process of making one operator and using it in many ways; it has one form but with many use cases. A good analogy is that different kinds of pens are all classified as pens, but their uses are vastly different. For example, there are pens for taking notes, and there are pens for drawing and creating artwork.
- *Inheritance*: This is the process of making a class from an existing class. Consider parents to children and children to parents; if you are born in a family and your parents are your birth parents, you and your siblings as children take or inherit both parents' characteristics. If you create a new class that derives from and is associated with an existing class, then the new class inherits the old class's characteristics.

Now that we have the basic OOP theory out of the way, let's use our routers and switches example to write classes to understand the OOP style of coding better. As your applications become more complex, the power of OOP truly shines as it binds the data. Let's take a look at a real example. Read and follow along in your Python interpreter.

Task

- 1 Let's write a class for routers and switches. For the most part, the routers and switches will have identical attributes; the difference is in the functions or the features. In this example, using inheritance, we can collapse or move the `def __init__` to the parent class. Let's quickly write our first classes.

Task

As you can see, both the router and switch classes have `serial-number`, `hostname`, `ipaddress (management)`, `modelnumber`, and `devicetype` attributes. The `__init__` is a reserved method in classes known as a *constructor* in OOP. The programmers read `__init__` as “dunder init dunder” and it allows the class to initialize a class’s attributes. The word `self` in the class is used to represent the class’s instance; with the `self` keyword, you can access the class’s attributes and methods. `self` is used to bind the attributes with the given arguments to the `@` syntax used in Java. It adds a special attribute to the class, method, and variable.

```
class Router:
    def __init__(self, serialnumber, hostname, ipaddress,
modelnumber, devicetype):
        self.serialnumber = serialnumber # 12 Hexadecimal
numbers
```

```
        self.hostname = hostname
```

```
        self.ipaddress = ipaddress # format 1.0.0.X -
254.254.254.XXX
```

```
        self.modelnumber = modelnumber
```

```
        self.devicetype = devicetype
```

```
    def process(self):
```

```
        print("Packet routing")
```

```
class Switch:
```

Task

```
def __init__(self, serialnumber, hostname, ipaddress,
modelnumber, devicetype):
```

```
    self.serialnumber = serialnumber # 12 Hexadecimal
numbers
```

```
    self.hostname = hostname
```

```
    self.ipaddress = ipaddress # format 1.0.0.X -
254.254.254.XXX
```

```
    self.modelnumber = modelnumber
```

```
    self.devicetype = devicetype
```

```
def process(self):
```

```
    print("Packet switching")
```

The router and switch both process packets; one does packet routing and the other packet switching, so a process was created to describe their functions. You have successfully created your first set of classes for your network devices.

2

Create a parent class called `Devices`, as shown next. The `Devices` class also takes the same attributes as the `Router` and `Switch` classes you created in task 1. So, create the `Devices` class in the same manner. You also want to add another function called `show` to let the function display the information when it is invoked.

```
>>> class Devices:
```

Task

```
... def __init__(self, serialnumber, hostname, ipaddress, modelnumber, devicetype):
```

```
...         self.serialnumber = serialnumber # 12
Hexadecimal numbers
```

```
...         self.hostname = hostname
```

```
...         self.ipaddress = ipaddress # format 1.0.0.X -
254.254.254.XXX
```

```
...         self.modelnumber = modelnumber
```

```
...         self.devicetype = devicetype
```

```
... def show(self): # show all the attributes
```

```
...     print(f"{self.serialnumber},{self.hostname},
{self.ipaddress},{self.modelnumber},
{self.devicetype}")
```

```
...
```

```
>>>
```

Now, create three classes called `Router`, `Switch`, and `Firewall`, respectively, and pass the class `Devices` to each device type class. Now all three classes will inherit the characteristics of the parent class, `Devices`. You can already see how classes and OOP simplify your code, because you did not have to create the same attributes three separate times. We have just created one for the parent class and applied it to all three child classes.

```
# Task
```

```
>>> class Router(Devices):
```

```
...     def process(self):
```

```
...         print("Packet routing")
```

```
...
```

```
>>> class Switch(Devices):
```

```
...     def process(self):
```

```
...         print("Packet switching")
```

```
...
```

```
>>> class Firewall(Devices):
```

```
...     def process(self):
```

```
...         print("Packet filtering")
```

```
...
```

Now create three variables , one for each child class as shown here:

```
>>> rt1 = Switch("001122AABBCC", "RT01", "1.1.1.1",  
"C4431-K9", "RT")
```

```
>>> sw1 = Router("001122DDEEFF", "SW02", "2.2.2.2",  
"WS-3850-48T", "SW")
```

Task

```
>>> fw1 = Firewall("003344ABCDEF", "FW02", "3.3.3.3",
"PA-5280", "FW")
```

From your Python interpreter, run the following commands to retrieve your device information. The `show()` method was automatically inherited from the parent class, `Devices`. This example was shown on the interpreter, but you can also write it into the code or download `oop1.py` and run it as a script. You have just finished studying the inheritance of OOP classes.

```
>>> rt1.show()
```

```
001122AABBCC,RT01,1.1.1.1,C4431-K9,RT
```

```
>>> sw1.show()
```

```
001122DDEEFF,SW02,2.2.2.2,WS-3850-48T,SW
```

```
>>> fw1.show()
```

```
003344ABCDEF,FW02,3.3.3.3,PA-5280,FW
```

- Next, let's build hierarchical classes so our comprehension of OOP solidifies, and we can use this in our real-life scenarios. The following example has the top-tier class called `Pynetauto_co`, which represents your company. The following is the child class called `Cisco`, representing the vendor of your company's networking devices. This is followed by the child class, `Devices`, and then under `Devices`, there are two child classes called `Router` and `Switch`. The inheritance is like a waterfall, so it is top-down. You can write this in Notepad or directly in the nano text editor.

oop_task1.py

```
# Task
```

```
class Pynetauto_co: # parent class of Cisco class
```

```
"Parent class of Cisco, HP, Juniper, Arista"
```

```
def __init__(self, companyname):
```

```
    self.companyname = companyname
```

```
def PrintInfo_P1(self):
```

```
    print("Pynetauto Company")
```

```
    #print("www.xyzptyltd.com")
```

```
class Cisco(Pynetauto_co): # parent class of Devices
```

```
class
```

```
"Parent class of Switch, Router and Firewall"
```

```
def __init__(self, vendor):
```

```
    self.Vendor = vendor
```

```
def PrintInfo_P2(self):
```

```
    print("Cisco")
```

```
    #print("www.cisco.com")
```

```
class Devices (Cisco): # parent class of Router and
```

```
Switch classes
```

```
"Parent class of Switch, Router and Firewall"
```

```
# Task
```

```
def __init__(self, serialnumber, hostname, ipaddress, modelnumber, devicetype):
```

```
    self.serialnumber = serialnumber # 12
    Hexadecimal numbers
```

```
    self.hostname = hostname
```

```
    self.ipaddress = ipaddress # format 1.0.0.X -
    254.254.254.XXX
```

```
    self.modelnumber = modelnumber
```

```
    self.devicetype = devicetype
```

```
def show(self): # show all the attributes
```

```
    print(f"{self.serialnumber},{self.hostname},
    {self.ipaddress},\n
```

```
    {self.modelnumber},{self.devicetype}")
```

```
class Router(Devices):
```

```
    def process(self):
```

```
        print("Packet switching")
```

```
class Switch(Devices):
```

```
    def process(self):
```

```
        print("Packet routing")
```

```
# Task
```

```
rt3 = Router("000111222222", "RT03", "2.2.2.2",  
"C4351/K9", "RT")
```

```
sw3 = Switch("000111222111", "SW03", "1.1.1.1", "WS-  
3850-48T", "SW")
```

```
rt3.PrintInfo_P1()
```

```
rt3.PrintInfo_P2()
```

```
rt3.show()
```

```
sw3.PrintInfo_P1()
```

```
sw3.PrintInfo_P2()
```

```
sw3.show()
```

When you save the previous OOP task script and run it from Python, it will return the following output :

```
Pynetauto Company
```

```
Cisco
```

```
000111222222,RT03,2.2.2.2,C4351/K9,RT
```

```
Pynetauto Company
```

```
Cisco
```

```
000111222111,SW03,1.1.1.1,WS-3850-48T,SW
```

OOP will bind the data together and empower you to write clean code. When you have to deal with many objects with similar attributes and cannot create each object's attributes, OOP will come to your rescue. Also, when we have to manage data in a dataframe or any SQL type, the OOP structure can help us better manage and handle data within our Python scripts. Now let's quickly move onto the next topic.

Flow Control and Controlling User Input: UID, PWD, and Information Collector

In this section, you will learn something fun and practical. While developing an interactive or noninteractive Python application, you often want to control an interactive user's data input. You want to make sure that the user input is precisely what is defined in your script. This, in turn, can be used to change the flow of your Python script. A straightforward example would be asking the user to enter yes or no. There could be four possible user input scenarios. In this case, first, type in yes or y; second, type in no or n; third, type something other than yes or no; and fourth, stare at the screen and type nothing in. Also, by introducing regular expressions into user input, you can home in to accept the information you want and reject any other variable values. You will see this in action in the next example, but first, let's have some fun with the yes/no user input scenario; then we'll reiterate the script to make it more useful, and study how it can change the flow of your Python script and how the reiteration can enhance your simple script.

The first script you will develop is a username and password input application using a yes/no script.

Task

- 1 The following is a simple yes/no user input example using the `if ~ else` statement. Create a new Python script called `yes_or_no1.py`, and run the script. There are a few flaws with this script, but we will enhance this first input script to make it useful in our real Python applications.

Create a separate directory on your server and create your first script as shown here:

Task

```
pynetauto@ubuntu20s1:~$ mkdir user_input
```

```
pynetauto@ubuntu20s1:~$ cd user_input
```

```
pynetauto@ubuntu20s1:~/user_input$ nano yes_or_no1.py
```

```
GNU nano
```

```
4.8 /home/pynetauto/user_input/yes_or_no1.py
```

```
def yes_or_no(): # Create function called yes_or_no
```

```
yes_or_no = input("Enter yes or no : ") # Ask for user input
```

```
yes_or_no = yes_or_no.lower() # change all input casing to lower casing
```

```
if yes_or_no == "yes": # if answer is 'yes' take the following action
```

```
    print("Oh Yes!")
```

```
else: # if answer is 'no' or other input, take the following action
```

```
    print("Oh No!")
```

```
yes_or_no() # Run the function
```

```
^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J
Justify      ^C Cur Pos
```

```
^X Exit      ^R Read File  ^\ Replace   ^U Paste Text ^T To
Spell  ^_ Go To Line
```

When you run the previous script and give input, the output should look like this for each response.

Task

This Python code is pretty basic. Think about where we can improve on this script.

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no1.py
```

```
Enter yes or no : Yes
```

Oh Yes!

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no1.py
```

```
Enter yes or no : no
```

Oh No!

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no1.py
```

```
Enter yes or no : maybe
```

Oh No!

2 The first example had the following flaws :

a. Not considering the abbreviated response from the user, `y` for yes and `n` for no.

b. No response and the incorrect response were treated as the same response.

Let's quickly correct the previous and see if we can reiterate this simple code and make it better.

Name your code file `yes_or_no2.py`.

```
pynetauto@ubuntu20s1:~/user_input$ nano yes_or_no2.py
```

Task

GNU nano

4.8

/home/pynetauto/user_input/**yes_or_no2.py**

```
def yes_or_no():
```

```
    yes_or_no = input("Enter yes or no : ")
```

```
    yes_or_no = yes_or_no.lower()
```

```
    if yes_or_no == "yes" or yes_or_no == "y": # Takes abbreviated response
```

```
        print("Oh Yes!")
```

```
    elif yes_or_no == "no" or yes_or_no == "n": # Takes abbreviated
response
```

```
        print("Oh No!")
```

```
    else: # Any other responses, print the following statement
```

```
        print("You have not entered the correct response.")
```

```
yes_or_no()
```

```
^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J Justify  ^C
Cur Pos
```

```
^X Exit      ^R Read File  ^\ Replace   ^U Paste Text ^T To Spell  ^_
Go To Line
```

The expected output examples are as shown here, but we can still improve this further :

Task

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no2.py
```

```
Enter yes or no : y
```

```
Oh Yes!
```

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no2.py
```

```
Enter yes or no : no
```

```
Oh No!
```

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no2.py
```

```
Enter yes or no : maybe
```

```
You have not entered the correct response .
```

You only want to allow `yes` or `no` as a response and want to prompt the user until a `yes`, `y`, `no`, or `n` response is provided. By restricting the users response, you are now dictating and controlling the flow of your script. How would you achieve this goal?

- 3 What part of the script do you have to change to get a `yes` or a `no` from the respondent? There are a few possible ways to achieve this, and as always, there are no right or wrong answers with Python coding, only the best recommendations. Let's look at the next example to achieve our goal.

In the following example, you will use a simple list with the expected answers, and you will only allow your script to run when those values are received as a response. Until you get the correct response, you will prompt the user that you expect a `yes` or `no` response .

```
pynetauto@ubuntu20s1:~/user_input$ nano yes_or_no3.py
```

Task

GNU nano

4.8

/home/pynetauto/user_input/**yes_or_no3.py**

```
def yes_or_no():
```

```
    yes_or_no = input("Enter yes or no : ")
```

```
    yes_or_no = yes_or_no.lower()
```

```
    expected_response = ['yes', 'y', 'no', 'n'] # Expected responses
```

```
    while yes_or_no not in expected_response: # Prompt until 'yes' or 'no'
        response is given
```

```
        yes_or_no = input("Expecting yes or no : ")
```

```
    if yes_or_no == "yes" or yes_or_no == "y": # 'yes' or 'y' action
```

```
        print("Oh Yes!")
```

```
    else: # 'no' or 'n' action
```

```
        print("Oh No!")
```

```
yes_or_no()
```

```
^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J Justify  ^C
Cur Pos
```

```
^X Exit      ^R Read File  ^\ Replace   ^U Paste Text ^T To Spell  ^_
Go To Line
```

Task

The expected output should look as follows, and the response must meet the criteria to pass the `while` loop. Otherwise, the user is asked over and over again for the correct response. Now, this looks quite promising, but where can you use this piece of code? In the next example, let's look at the real example and how it is used.

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no3.py
```

```
Enter yes or no : Y
```

```
Oh Yes!
```

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no3.py
```

```
Enter yes or no : NO
```

```
Oh No!
```

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no3.py
```

```
Enter yes or no : maybe
```

```
Expecting yes or no : maybe
```

```
Expecting yes or no : yes
```

```
Oh Yes!
```

- The following script collects two sets of network administrator ID and password. To start, get the first administrator's credentials, and then ask if the second
- 4 administrator's credentials are the same. If the response is `yes` or `y`, the user ID and passwords are the same. If the response is `no` or `n`, then collect the second set of user ID and password .

Task

```
pynetauto@ubuntu20s1:~/user_input$ nano yes_or_no4.py
```

```
GNU nano 4.8 /home/pynetauto/user_input/  
yes_or_no4.py
```

```
from getpass import getpass
```

```
def get_credentials():
```

```
    #Prompts for, and returns a username1 and password1
```

```
    username1 = input("Enter Network Admin1 ID: ") # Request for username 1
```

```
    password1 = getpass("Enter Network Admin1 PWD: ") # Request for pass-  
word 1
```

```
    print("Username1 :", username1, "Password1 :", password1) # Print user-  
name and password
```

```
    #Prompts for username2 and password2
```

```
    yes_or_no = input("Network Admin2 credentials same as Network Admin1  
credentials? (Yes/No): ").lower() # Ask if Network Admin 2 has the same  
credentials as Admin 1
```

```
    expected_response = ['yes', 'y', 'no', 'n'] # Expect any of these four  
responses
```

```
    while yes_or_no not in expected_response: # Prompt until 'yes' or 'no'  
response is given
```

```
        yes_or_no = input("Expecting yes or no : ")
```

Task

```
if yes_or_no == "yes" or yes_or_no == "y": # If 'yes' or 'y', credentials are the same as Admin1
```

```
username2 = username1
```

```
password2 = password1
```

```
print("Username2 :", username2, "Password2: ", password2) # Print username and password
```

```
else: # If 'no' or 'n', request for Admin2 username and password
```

```
username2 = input("Enter Network Admin2 ID : ")
```

```
password2 = getpass("Enter Network Admin2 Password : ")
```

```
print("Username2 :", username2, "Password2 :", password2) # Print username and password
```

```
get_credentials()
```

```
^G Get Help  ^O Write Out    ^W Where Is   ^K Cut Text   ^J Justify    ^C
Cur Pos
```

```
^X Exit      ^R Read File   ^\ Replace    ^U Paste Text ^T To Spell   ^_
Go To Line
```

Run the script to test the user ID and password collector script. The expected output should be similar to the following output. Do you think you can reiterate this script and improve this script? Let's see the final example.

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no4.py
```

Task

```
Enter Network Admin1 ID: hugh
```

```
Enter Network Admin1 PWD:*****
```

```
Username1 : john Password1 : password1
```

```
Network Admin2 credentials same as Network Admin1 credentials? (Yes/No):  
yes
```

```
Username2 : john Password2: password1
```

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no4.py
```

```
Enter Network Admin1 ID: hugh
```

```
Enter Network Admin1 PWD: *****
```

```
Username1 : hugh Password1 : password1
```

```
Network Admin2 credentials same as Network Admin1 credentials? (Yes/No):  
no
```

```
Enter Network Admin2 ID : john
```

```
Enter Network Admin2 Password : *****
```

```
Username2 : john Password2 : password777
```

- 5 In this final reiteration of the `yes` or `no` example, you will add a password verification feature to make sure that the user's first password is verified with the second password. Both passwords must match; the `getpass` module from the `getpass` library hides the password while the user is typing the password. By adding the password verification feature, you are minimizing the likelihood of an incorrect password.

Task

```
pynetauto@ubuntu20s1:~/user_input$ nano yes_or_no5.py
```

```
GNU nano
```

```
4.8 /home/pynetauto/user_input/yes_or_no5.py
```

```
from getpass import getpass
```

```
def get_credentials():
```

```
    #Prompts for, and returns a username1 and password1
```

```
    username1 = input("Enter Network Admin1 ID: ") # Request for username 1
```

```
    password1 = None # Set password1 to None (initial value to None)
```

```
    while not password1: # Until password1 is given
```

```
        password1 = getpass("Enter Network Admin1 PWD : ") # Get password1
```

```
        password1_verify = getpass("Confirm Network Admin1 PWD : ") # Request  
for validation
```

```
        if password1 != password1_verify: # If the password1 and verification  
password does not match
```

```
            print("Passwords do not match. Please try again.") # Print this  
information
```

```
        password1 = None # Set the password to None and ask for password1  
again
```

```
    print("Username1 :", username1, "Password1 :", password1) # Print  
username and password
```

Task

```
#Prompts for username2 and password2
```

```
yes_or_no = input("Network Admin2 credentials same as Network Admin1
credentials? (Yes/No): ").lower() # Ask if Network Admin 2 has the same
credentials as Admin 1
```

```
expected_response = ['yes', 'y', 'no', 'n'] # Expect any of these four
responses
```

```
while yes_or_no not in expected_response: # Prompt until 'yes' or 'no'
response is given
```

```
yes_or_no = input("Expecting yes or no : ")
```

```
if yes_or_no == "yes" or yes_or_no == "y": # If 'yes' or 'y', creden-
tials ate the same as Admin1
```

```
username2 = username1
```

```
password2 = password1
```

```
print("Username2 :", username2, "Password2 :", password2) # Print
username and password
```

```
else: # If 'no' or 'n', request for Admin2 username and password
```

```
username2 = input("Enter Network Admin2 ID: ") # Request for username
2
```

```
password2 = None # Explanation same as above
```

```
while not password2: # Explanation same as above
```

Task

```
password2 = getpass("Enter Network Admin2 PWD : ") # Explanation
same as above
```

```
password2_verify = getpass("Confirm Network Admin2 PWD : ")
```

```
if password2 != password2_verify: # Explanation same as above
```

```
print("Passwords do not match. Please try again.")
```

```
password2 = None # Explanation same as above
```

```
print("Username2 :", username2, "Password2 :", password2) # Print
username and password
```

```
get_credentials()
```

```
^G Get Help  ^O Write Out    ^W Where Is  ^K Cut Text   ^J Justify   ^C
Cur Pos
```

```
^X Exit      ^R Read File   ^\ Replace   ^U Paste Text ^T To Spell  ^_
Go To Line
```

You can learn the process of reiterating code to improve the original code. Once you are happy with your code's quality, you can save it to your code repository and document it, or you can also choose to share it with others on your team or on the online community. In reality, Python coding is not always fun as you have to get anal about everything you code. You frequently have to adjust the reiteration of your code and other code. Learn to start with an elementary script and then make improvements through reiterations.

Here is the expected output :

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no5.py
```

Task

```
Enter Network Admin1 ID: hugh
```

```
Enter Network Admin1 PWD :*****
```

```
Confirm Network Admin1 PWD : ***** # incorrect password typed
```

```
Passwords do not match. Please try again.
```

```
Enter Network Admin1 PWD : *****
```

```
Confirm Network Admin1 PWD : ***** # correct password typed
```

```
Username1 : hugh Password1 : password123
```

```
Network Admin2 credentials same as Network Admin1 credentials? (Yes/No):  
yes
```

```
Username2 : hugh Password2 : password123
```

```
pynetauto@ubuntu20s1:~/user_input$ python3 yes_or_no5.py
```

```
Enter Network Admin1 ID: john
```

```
Enter Network Admin1 PWD : *****
```

```
Confirm Network Admin1 PWD : *****
```

```
Username1 : john Password1 : password777
```

```
Network Admin2 credentials same as Network Admin1 credentials? (Yes/No):  
no
```

```
Enter Network Admin2 ID: bill
```

Task

```
Enter Network Admin2 PWD : *****
```

```
Confirm Network Admin2 PWD : *****
```

```
Username2 : bill Password2 : password888
```

You have completed an exercise to write a basic Python application and then improve the code by adding new features and ideas through reiterations. Your first code is never perfect; it takes time and imagination to make even the most straightforward Python programs for your work. Next, let's prepare for the final lab and plan for IOS upgrade application development by discussing the actual IOS upgrade tasks and processes.

Lab Preparation

One problem with IOS upgrade tests on GNS3 is that it does not fully support the reloading feature within GNS3. This is one of the downfalls of emulation software compared to real IOS software. There are two possible ways we can start developing and testing IOS upgrade scenarios. The first way is to purchase the real hardware and test, and the second way is to find an alternative or virtual IOS image that supports upgrading IOS in a virtual environment. From the first chapter of this book, I have promised you that everything we do in this book can be emulated and practiced without expensive Cisco devices. Also, all learning will be completed from a single PC or laptop. However, in developing an application to upgrade a Cisco IOS, I have to admit there is no replacement for the actual hardware equipment. A virtualized Cisco router or a switch is not the same as a physical router or a switch. The virtual routers and switches have logical parts as in files but without the real motherboard, pluggable ports, power-consuming power supply, and noisy cooling fans. Also, some parts of switching can be emulated, as you have seen in the use of IOSvL2 switches. These switches are not 100 percent like production equipment as they miss the real ASCII support for LAN switching. We can use the closest Cisco IOS device on a single laptop to emulate the IOS upgrade using the virtual cloud router, Cisco CSR 1000v. As long as you can get your

hands on a copy of Cisco CSR 1000v, you can still practice the IOS upgrading scenario using Python scripts. Unfortunately, I cannot share this software with every reader, if you can get your own copy, you will be able to follow the whole lab from beginning to end. If you cannot get ahold of the CSR 1000v IOS XE images or similar images used here, you may have to purchase a couple of very outdated Cisco 2621XM/2651XM routers for \$10 each on eBay and emulate the reloading section of the code.

CSR 1000v IOS XE Software and Download

To prepare for this lab, we have to download a set of Cisco CSR 1000v IOS XE images. Pay close attention and take notes on the different file extensions of the set of files we are about to download for our final lab. For virtual machine (router) creation, we first need to download the file ending with the .ova file extension. The VMware ESXi6.5 virtual machine template file and the IOS upgrading practice need the latest IOS XE file with the file extension .bin. You can find more details on each of these files in Table 17-1.

Table 17-1. CSR 1000v IOS XE Files Used in This Chapter

Item	From IOS XE Detail	To IOS XE Detail
Release Date	30-Nov-2018	04-Sep-2020
Release	Fuji-16.7.3	Fuji-16.9.6
FileName	csr1000v-universalk9.16.07.03.ova	csr1000v-universalk9.16.09.06.SPA.bin
Release Date	30-Nov-2018	04-Sep-2020
Min Memory	DRAM 4096 Flash 8192	DRAM 4096 Flash 8192
Size	397.99 MB	416.35 MB
MD5	aa6cba7ff85afb3e7ca29831a28aefb4	77878ae6db8e34de90e2e3e83741bf39

As you have noticed, the minimum memory and flash size requirements are quite demanding to run on GNS3, and the GNS3 will struggle to run Cisco CRS 1000v routers in GNS3 environment. Since network automation's power is realized only when the same application can run on more than a single device, you will be creating two virtual CSR 1000v routers on your computer. That means the memory consumption from these two virtual routers alone will be 8 GB combined. Also, as mentioned, on the basic GNS3 configuration, the rebooting emulation does not have the expected behaviors as you would expect from a real production environment, so rather than building the IOS XE routers on GNS3, you will be importing the `.ova` file and creating two Cisco IOS XE routers on VMware Workstation 15 Pro. Then, you can follow the IOS upgrade by uploading the latest `.bin` file of the same IOS XE train. Since you will not be testing any serious routing concepts but only required to emulate IOS upgrade, this lab setup is sufficient and helpful to develop a real working Python application for end-to-end IOS upgrade scenarios.

If you have an active Cisco service contract and have access to CCO, please log in and download the following or similar sets of files for your lab practice. It does not have to be Fuji. If you prefer another IOS XE flavor for CSR 1000v, you should download a set of files for this lab. See Figure [17-1](#).

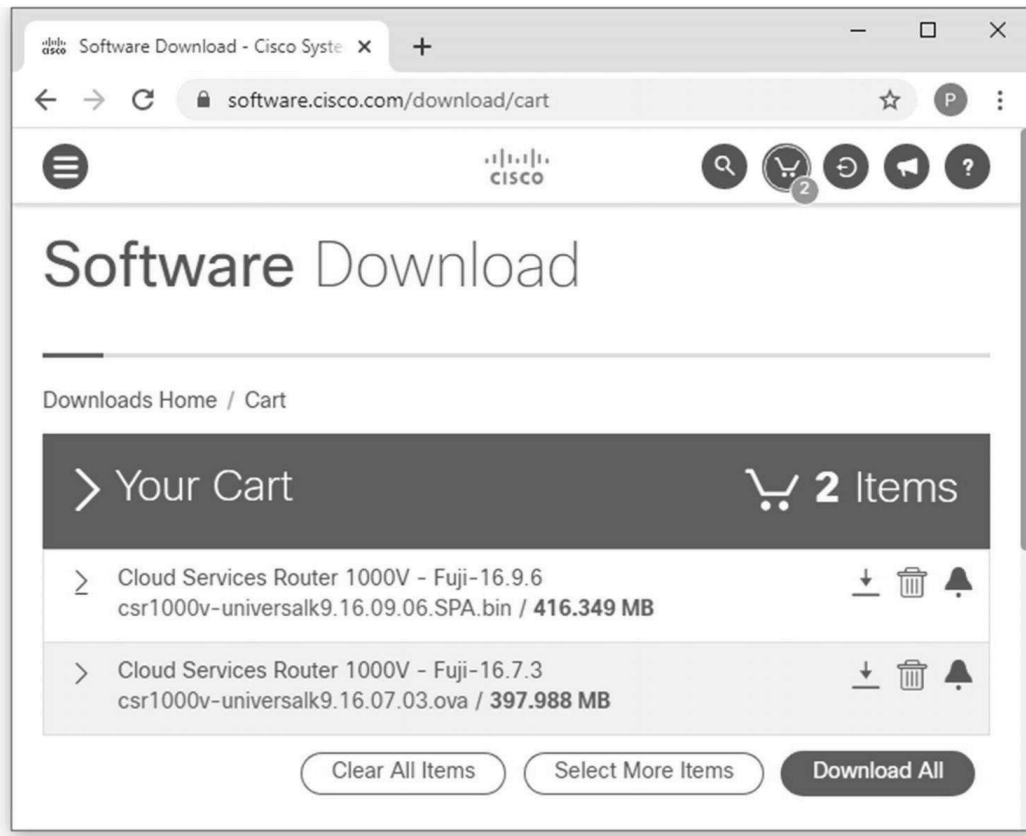


Figure 17-1. CSR 1000v IOS XE downloads from Cisco software download site

Cisco CSR 1000v Installation on VMware Workstation

Installing CSR 1000v on VMware Workstation 15 is no different from installing the GNS3 VM .ova file you previously installed. But this time, you are going to create two virtual machines instead of one. Also, for this lab, you can keep the GNS3 completely powered off, but you will still need the Python server, ubuntu20s1, powered on. As soon as we build the virtual routers, we will configure both routers with the minimal configuration and get on with the IOS upgrade application development. See Figure 17-2.

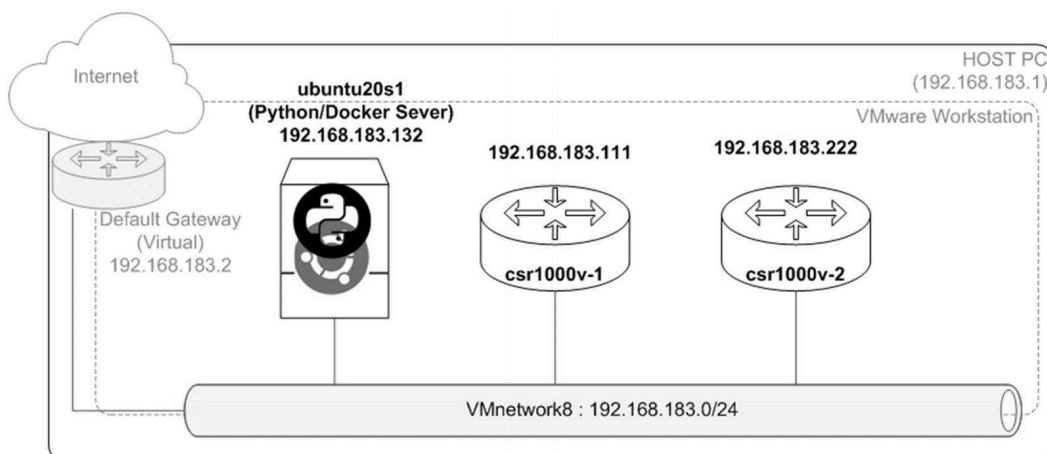


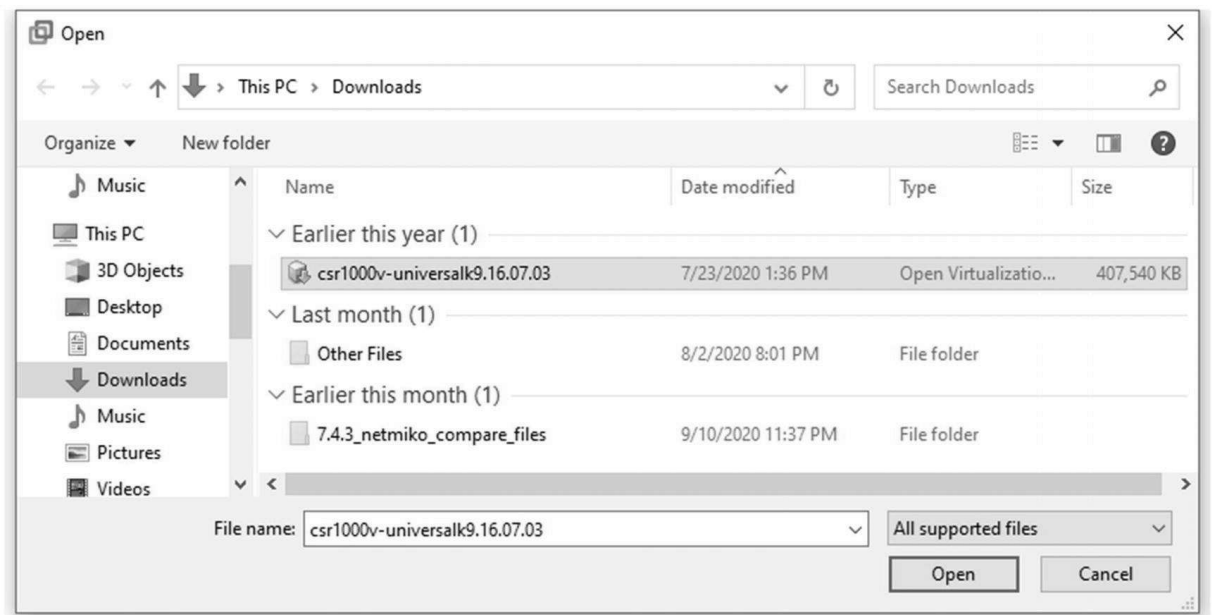
Figure 17-2. Chapter 10 lab equipment

Now follow these tasks to install both `csr1000v-1` and `csr1000v-2` virtual routers. The installation of `csr100-v` is straightforward.

#	Task
---	------

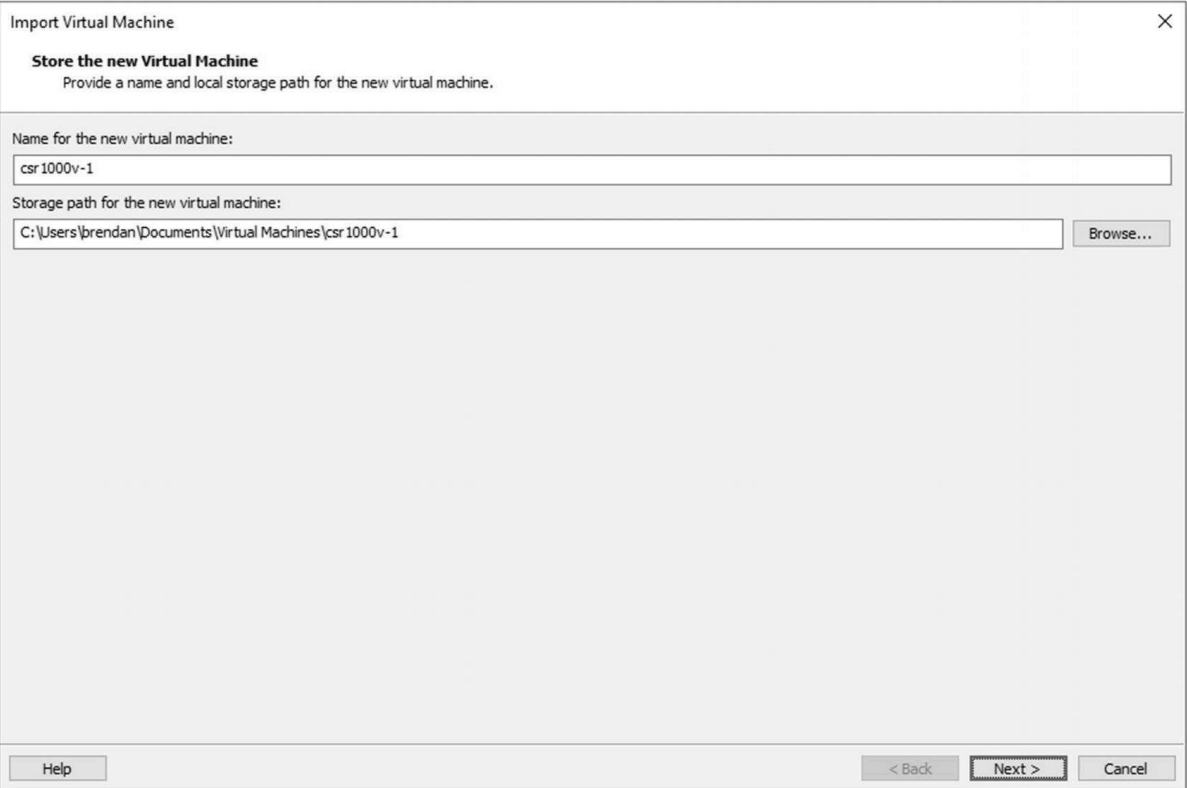
To create the virtual machine, first open the VMware Workstation main window and go to File ► Open. Then go to the Downloads folder and select the `csr1000v-universalk9.16.07.03.ova` file. See Figure 17-3.

1

**Figure 17-3.** VMware Workstation, selecting the `csr1000v.ova` file

- 2 For the first virtual router, give `csr1000v-1` as the name of this device. When you create the second virtual router, you just need to change the last digit of the router name to 2, so it would be `csr1000v-2`. See Figure 17-4.

Task



Import Virtual Machine

Store the new Virtual Machine
Provide a name and local storage path for the new virtual machine.

Name for the new virtual machine:
csr1000v-1

Storage path for the new virtual machine:
C:\Users\brendan\Documents\Virtual Machines\csr1000v-1 Browse...

Help < Back Next > Cancel

Figure 17-4. csr1000v-1, giving the new router name

- 3 Leave the Deployment Options setting as Small and click the Next button. See Figure [17-5](#).

Task

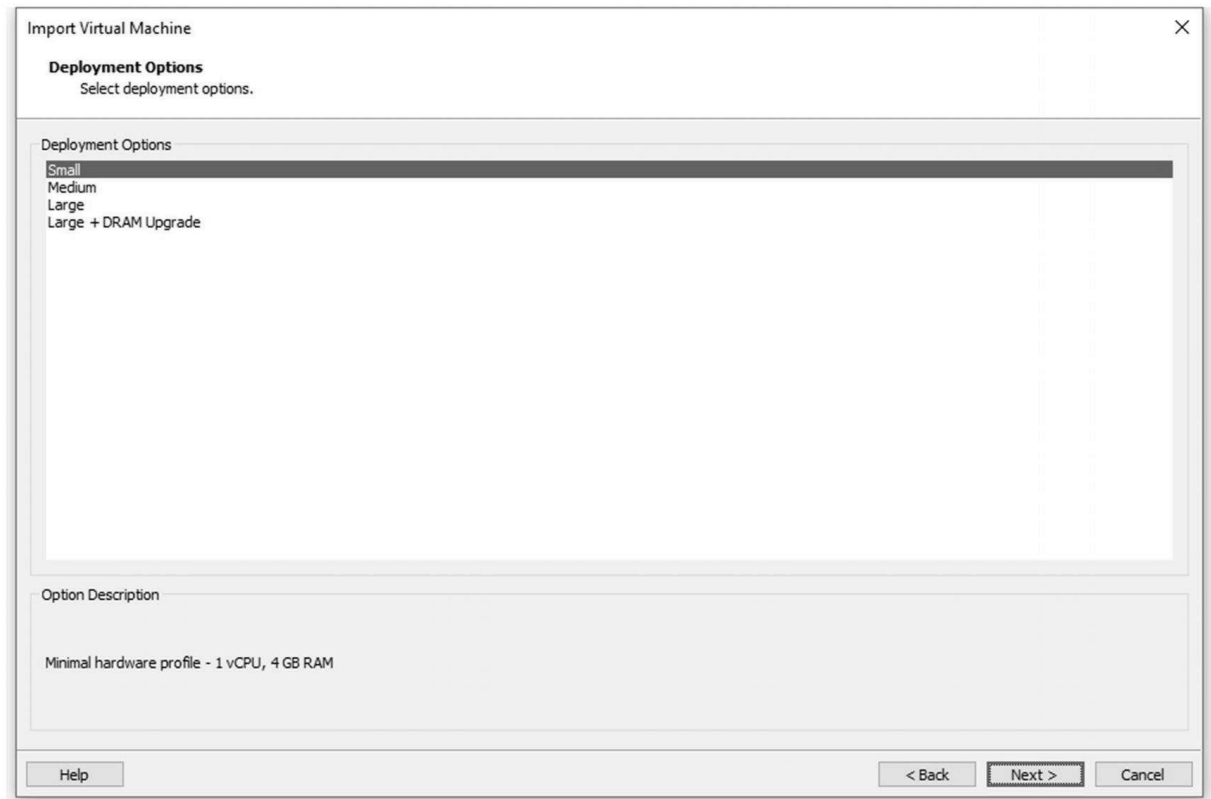


Figure 17-5. csr1000v-1, deployment options

- Now to import the virtual router, you are going to fill out the following properties.
- 4 Configuring the properties on the GUI is the same as configuring these settings through the CLI.

For 1. Bootstrap Properties , fill in the Router Name, Login Username, and Login Password fields. See Figure **17-6**.

Task

Import Virtual Machine

Properties
Additional properties for this Virtual Machine.

1. Bootstrap Properties
2. Features
3. Additional Configuration Properties
4. Intercloud Configuration Properties:

1. Bootstrap Properties

Router Name: csr1000v-1

Login Username: pynetauto

Login Password: ••••••••

Confirm Login Password: ••••••••

Management Interface: GigabitEthernet1

Management VLAN:

Management Interface IPv4 Address/Mask:

Management IPv4 Gateway:

Management IPv4 Network:

PNSC IPv4 Address:

PNSC Agent Local Port:

PNSC Shared Secret Key:

Confirm PNSC Shared Secret Key:

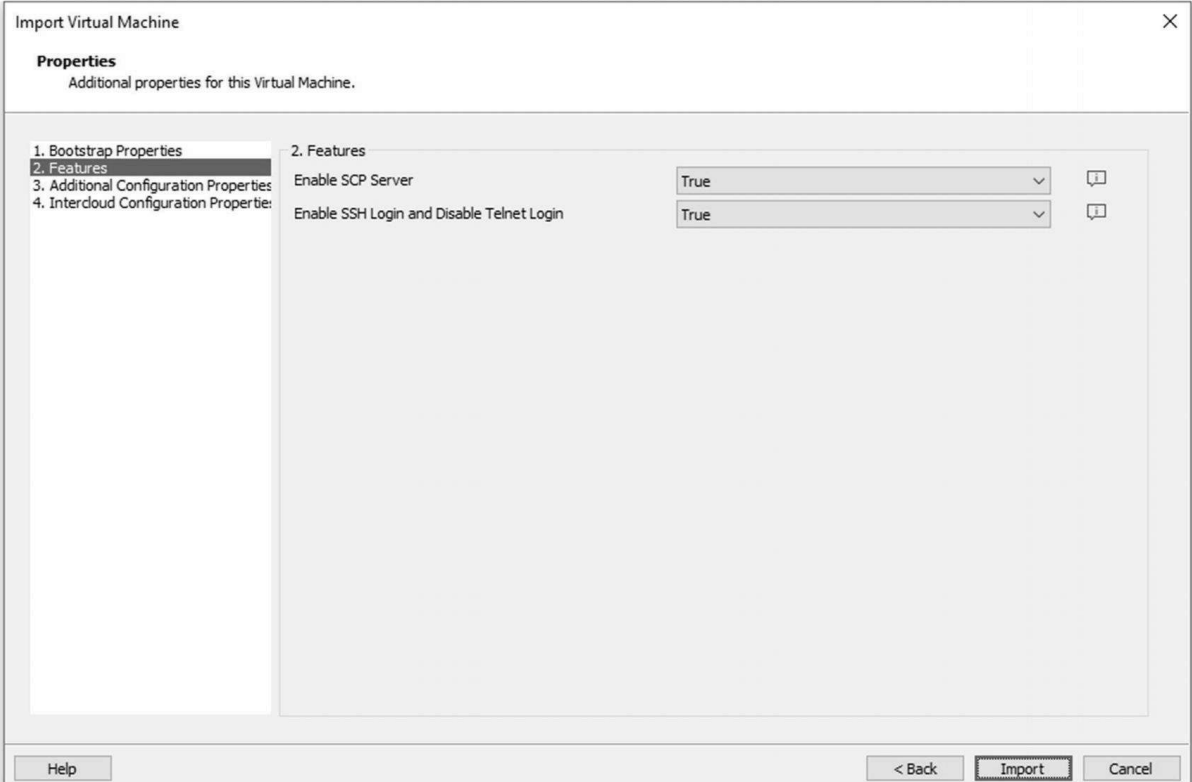
Remote Management IPv4 Address (optional, deprecated):

Help < Back Import Cancel

Figure 17-6. csr1000v-1, 1. Bootstrap Properties

For 2. Features , change False to True for both Enable SCP Server and Enable SSH and Disable Telnet Login. See Figure [17-7](#).

Task



The screenshot shows a window titled "Import Virtual Machine" with a close button (X) in the top right corner. Below the title bar is a section labeled "Properties" with the subtitle "Additional properties for this Virtual Machine." On the left side, there is a list of four property categories: "1. Bootstrap Properties", "2. Features" (which is selected and highlighted), "3. Additional Configuration Properties", and "4. Intercloud Configuration Properties". To the right of this list, under the "2. Features" heading, there are two configuration items: "Enable SCP Server" and "Enable SSH Login and Disable Telnet Login". Each item has a dropdown menu set to "True" and a small information icon (i) to its right. At the bottom of the window, there are three buttons: "Help", "< Back", and "Import" (which is highlighted), and a "Cancel" button on the far right.

Figure 17-7. csr1000v-1, 2. Features

For 3. Additional Configuration Properties , fill in Enable Password and a testing domain name. See Figure [17-8](#).

No configuration is required for 4. Intercloud Configuration Properties; leave the settings at the defaults.

Task

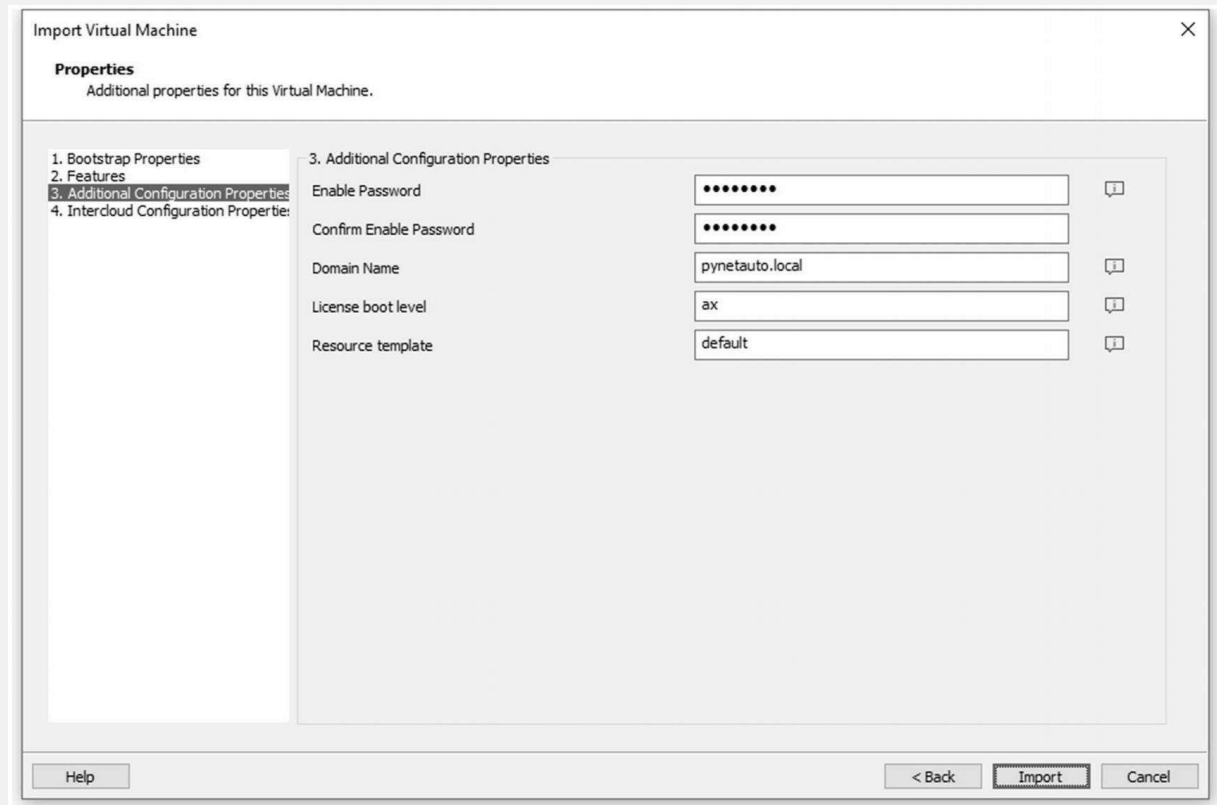


Figure 17-8. csr1000v-1, 3. Additional Configuration Properties

- 5 Double-click the screen (or press Ctrl+G) and hit any key to continue the router import. Press Ctrl+Alt to move back to the host Windows machine. Press any key to continue when you see the following message on the screen:

```
LoLoading stage2 .....
```

```
Press any key to continue
```

```
ading stage2 .....
```

- 6 You may have to wait for three to five minutes for the virtual machine to be imported to the workstation. Given enough time, the router import will be completed as shown next. As soon as the importing completes, log in and check the interface status .

```
Ecsr1000v-1>enable
```

#Task

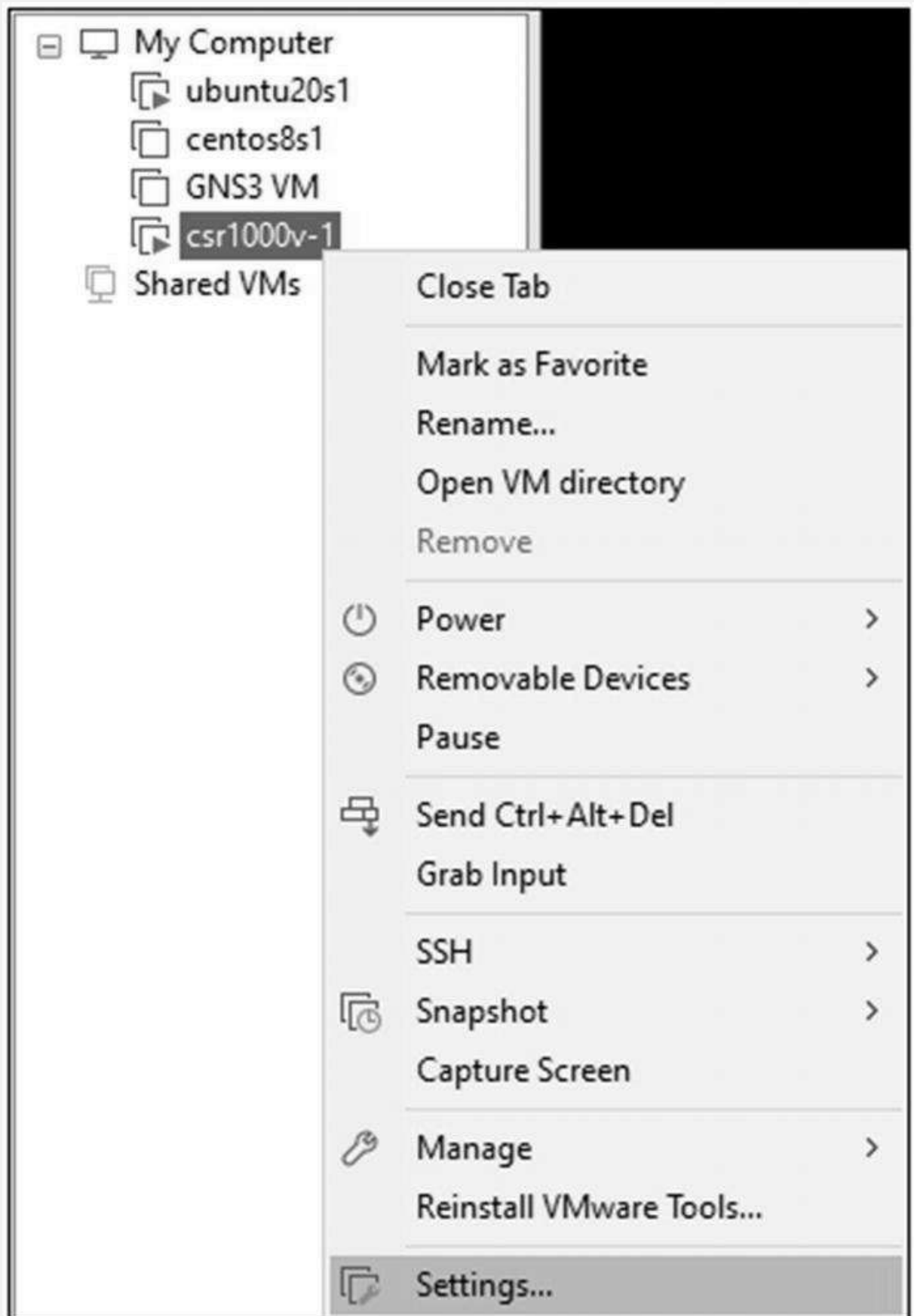
Password :

csr1000v-1#show ip interface brief

Interface	IP- Address	OK?	Method	Status	Protocol	
GigabitEthernet1	unassigned	YES	unset	administratively	down	down
GigabitEthernet2	unassigned	YES	unset	administratively	down	down
GigabitEthernet3	unassigned	YES	unset	administratively	down	down
			S	unset	administratively	down
						down

7 Right-click the new virtual router, `csr1000v-1`, and select **Settings** . The network adapter type needs to be changed. See Figure [17-9](#).

Task

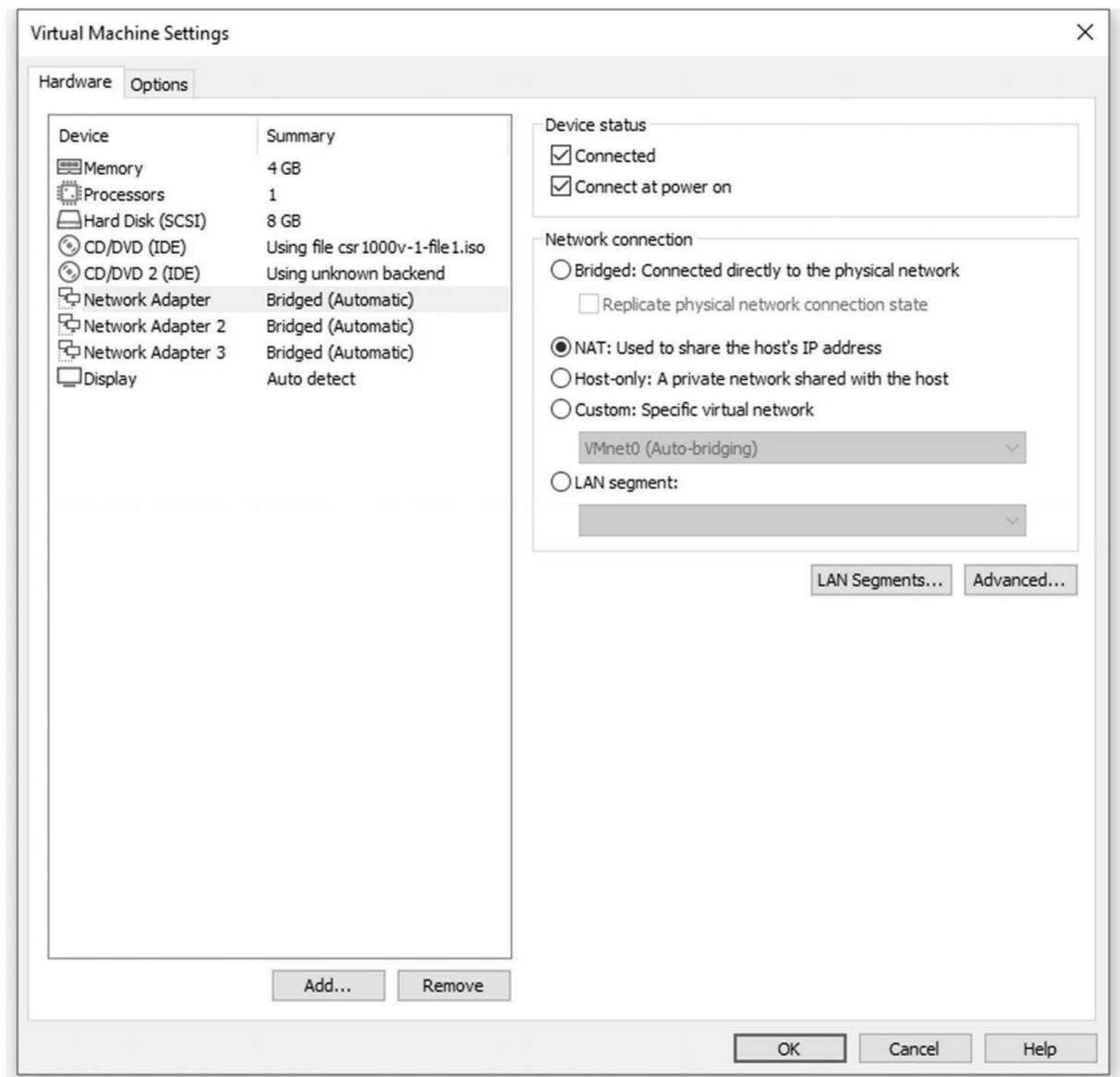


Task

Figure 17-9. csr1000v-1, opening Settings ...

Under Hardware, change the Network Adapter configuration from Bridged (Automatic) to “NAT: Used to share the host’s IP address.” See Figure [17-10](#).

8

**Figure 17-10.** csr1000v-1, changing Network Adapter (GigabitEthernet1) to NAT

9 When the virtual router build completes, the interfaces will be in an administrative down/down state like all other Cisco routers. Before bringing up the interface, let’s quickly assign the correct IP address for the GigabitEthernet1 interface with

Task

192.168.183.111/24. Since we need only one interface for our IOS upgrade tool development, you do not have to configure the other interfaces .

```
csr1000v-1#config terminal
```

```
csr1000v-1 (config) #interface GigabitEthernet1
```

```
csr1000v-1 (config-if) #ip address 192.168.183.111 255.255.255.0
```

```
csr1000v-1 (config-if) #no shut
```

```
csr1000v-1 (config-if) #end
```

Note that once you create the second virtual router, you should configure the GigabitEthernet1 interface with 192.168.183.222/24.

```
csr1000v-2#config terminal
```

```
csr1000v-2 (config) #interface GigabitEthernet1
```

```
csr1000v-2 (config-if) #ip address 192.168.183.222 255.255.255.0
```

```
csr1000v-2 (config-if) #no shut
```

```
csr1000v-2 (config-if) #end
```

- 10** On your Windows host, open the PuTTY SSH client and log into `csr1000v-1` (192.168.183.111). When you are prompted with PuTTY security alert, make sure you click the Yes button to accept the device's host key. Also, don't forget to send some ICMP commands to the `ubuntu20s1` server to test the connectivity. See Figure [17-11](#).

Task

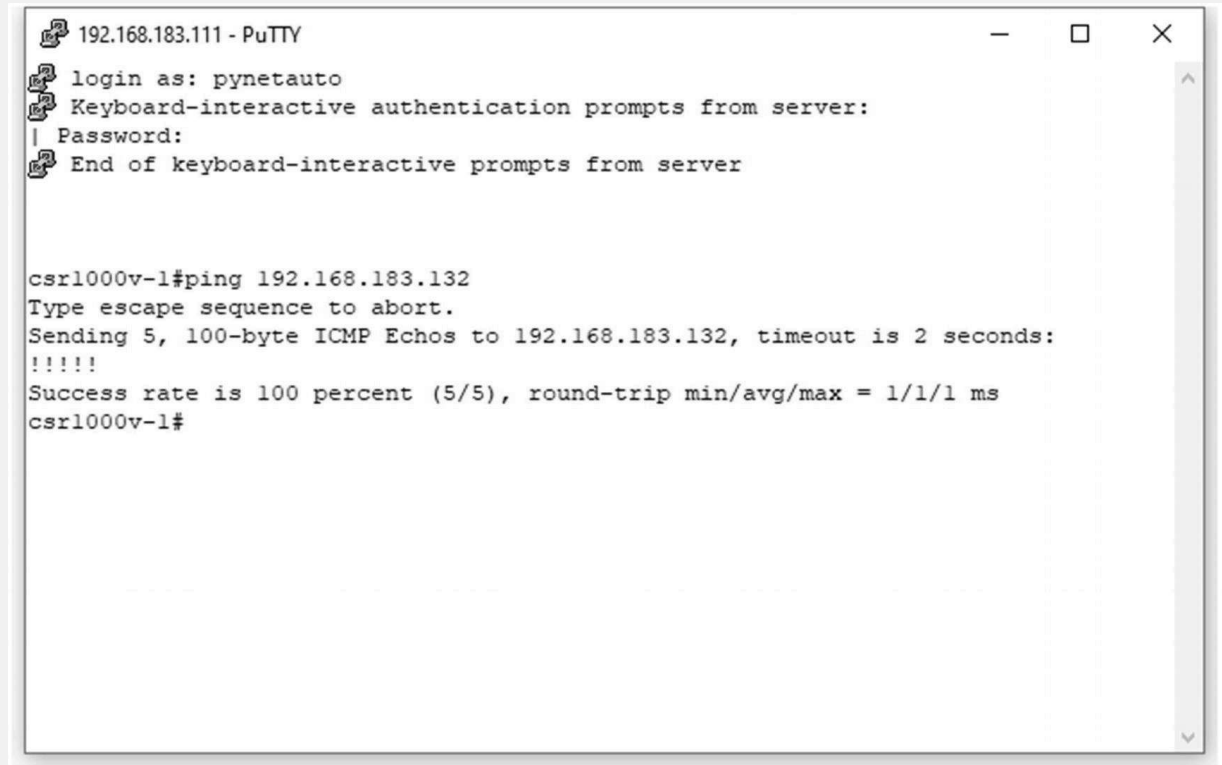


Figure 17-11. csr1000v-1, PuTTY login for the first time

11 To complete the basic configuration, let's check the domain and create the RSA key for secure key exchange. Also, disable the timeout for VTY lines to stop the SSH session timeouts; only use this feature for the lab environment. Save the configuration by running `write memory`.

```
csr1000v-1#show run | in ip domain
```

```
ip domain name pynetauto.local
```

```
csr1000v-1#conf terminal
```

Enter configuration commands, one per line. End with CNTL/Z.

```
csr1000v-1(config)#crypto key generate rsa
```

The name for the keys will be: csr1000v-1.pynetauto.local

Task

Choose the size of the key modulus in the range of 360 to 4096 for your

General Purpose Keys. Choosing a key modulus greater than 512 may take

a few minutes.

How many bits in the modulus [512]: **1024**

% Generating 1024 bit RSA keys, keys will be non-exportable...

[OK] (elapsed time was 0 seconds)

```
csr1000v-1(config)#line vty 0 15
```

```
csr1000v-1(config-line)#exec-timeout 0
```

```
csr1000v-1(config-line)#end
```

```
csr1000v-1#write memory
```

Building configuration...

[OK]

- 12** Once you are happy with the basic configuration , take a snapshot of the virtual router, as shown in Figure **17-12**.

Task

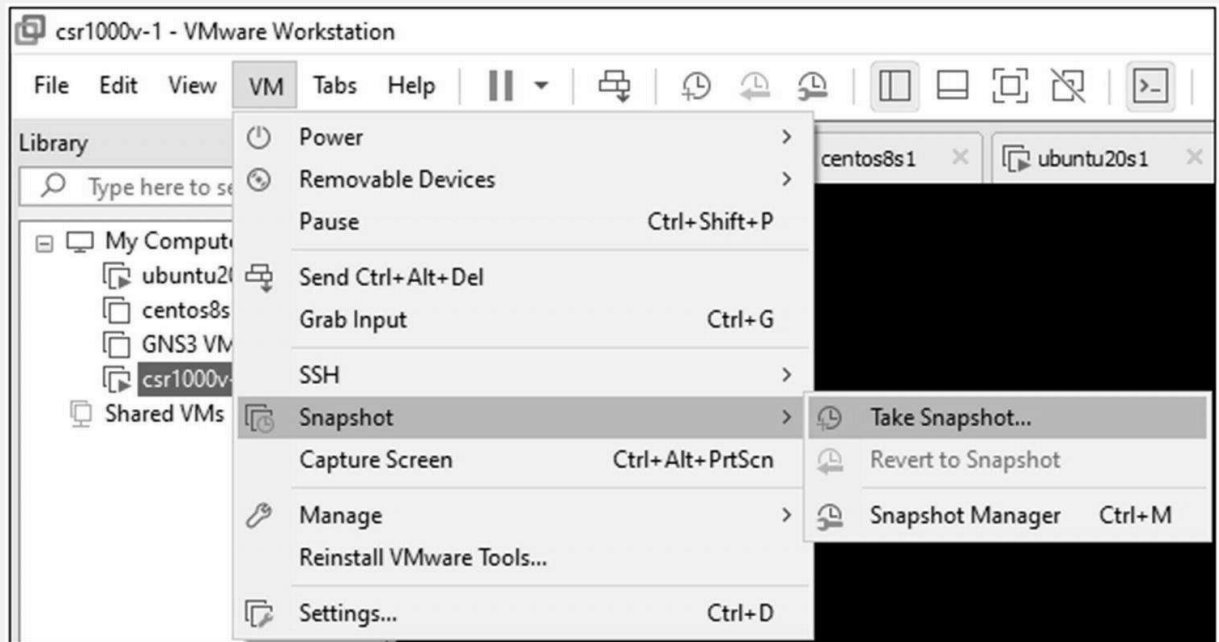


Figure 17-12. csr1000v-1, taking a snapshot

- 13 Now, repeat steps 1 through 12 to create the second virtual router (csr1000v-2) with the following details:

Second router name: **csr1000v-2**

Second router IP: **192.168.183.222 255.255.255.0**

- 14 Once you have two csr1000v routers (csr1000v-1 and csr1000v-2) installed, then you are ready to move on to the Cisco IOS upgrade processes . See Figure [17-13](#).

Task

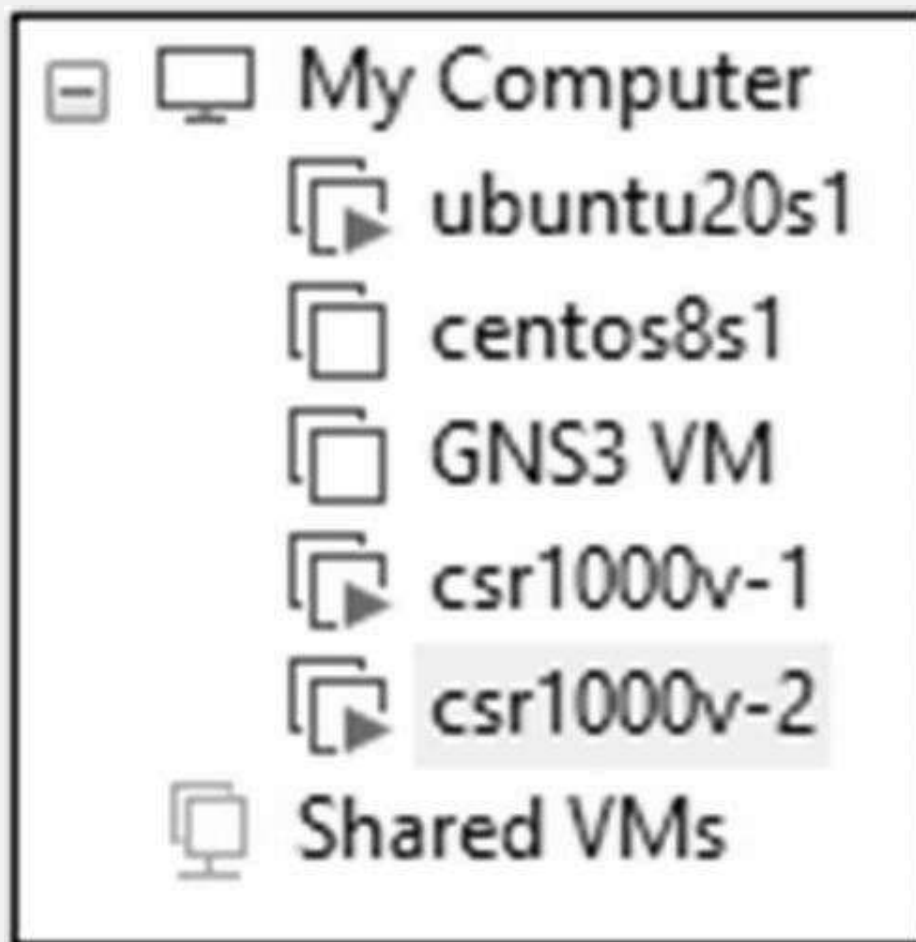


Figure 17-13. csr1000v-1 and csr1000v-2 up and running

You have now completed installing two Cisco CSR 1000v routers for IOS upgrade application development. Let's quickly go over the process of upgrading the Cisco IOS devices.

Discussion of How an IOS (IOS-XE/XR) Is Upgraded on Cisco Devices

Upgrading Cisco IOS (IOS/IOS-XE/IOS-XR) on Cisco routers and switches is not for faint-hearted engineers when hundreds and thousands of business-critical applications are operating during business hours as well as out of business hours. Even when the system and DevOps engineers go home, the enterprise routers and switches must provide IP connectivity to end users and IP services to other systems, allowing the backup services to run in the middle of the night. Enterprise networking devices are

operational almost 24/7/365, and this is why you have to buy reputable vendor products (such as Cisco) for enterprise networking. If any services fail, then the first finger pointing goes to IP gateway (routers) devices, even for company email, cloud, or system backup issues. In an ITIL-driven organization, you always have to go through change control with a lot of red tape to perform an IOS upgrade. The reasons for upgrading IOS devices are many. The most common are outdated IOS, bugs, Cisco TAC's recommendation, or third-party agreements. The IOS upgrade commitment is written into the support contract, and IOS is upgraded as part of network patch management.

Tasks Involved in a Cisco IOS Upgrade

Let's quickly look at how a seasoned network engineer would upgrade an outdated IOS to the latest IOS version. The IOS upgrade process involves a predefined set of tasks, but there will be varying differences in the pre- and post-checks from one engineer to another. However, most of the time, the IOS upgrade process is relatively standardized but requires a lot of attention and takes place outside of business hours to avoid major outages. You are only allowed to perform IOS upgrades after midnight with a small change window in extreme cases. This type of change usually occurs at odd hours, which contributes to insomnia among long-time network engineers and puts a lot of pressure on marriages in the long term. Anyway, network automation is good news for both network engineers and their families.

Let's study a general IOS upgrade workflow that an engineer needs to follow to complete an IOS upgrade on Cisco routers or switches. Figure **17-14** illustrates a general Cisco IOS upgrade workflow for many of its common platforms.

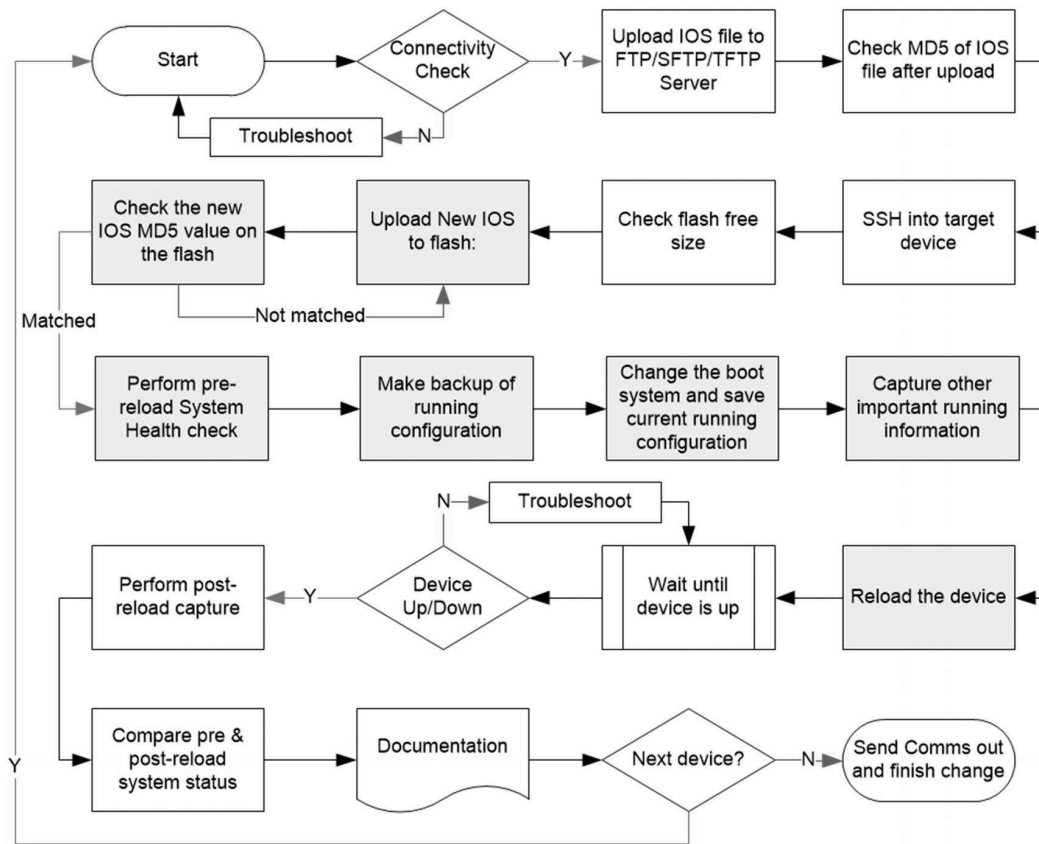


Figure 17-14. General Cisco IOS upgrade workflow

As depicted in Figure 17-14, the Cisco IOS upgrade process is quite involved; in other words, it's labor-intensive. If there is more than one device to upgrade, then the whole process immediately becomes repetitive. So, why not automate an engineer's tasks? In the final lab in the final two chapters, you will be guided to develop small Python modules to replace most of the tasks depicted in Figure 17-14.

Let's break down Figure 17-14 into three significant stages and many linear steps to better understand what tasks must be translated and written into the Python code. Here you really must wear an application developer/programmer's hat while keeping your network engineer's hat on. Table 17-2 provides the manual task-driven Cisco IOS upgrade breakdown ; this process may be slightly different from one generation to another and from one platform to another. In general, the principles are almost identical.

Table 17-2. IOS Upgrade Task Breakdown

Phase 1: Pre-check

- 1 Check the network connectivity between the server and network devices.

Phase 1: Pre-check

2 Collect the user's login credentials and also user input as variables.

3 Check the MD5 value of new IOS as a variable.

- Compare the user's MD5 value versus the server's checked MD5 value .

4 Check the flash size on the Cisco device as a variable.

- Give the user an option to delete an old IOS file on `flash:/.`
- Give an option to look inside a directory on a Cisco switch and delete an old IOS file.

5 Make backups of `running-config`.

Phase 2: IOS Uploading and Pre-uploading Check

6 Upload the new IOS file from a file server to the router's flash.

7 Check the new IOS MD5 value on the Cisco device's flash.

- Compare the server's MD5 value versus the switch MD5 value.

8 Give the user a review of the result.

- Give the user an option to reload the device at a deferred time if required.

9 Before device reload:

- Change the boot system and save the current `running-config` to `start-up config`.

Phase 1: Pre-check

- Take backups of the running configuration on local/TFTP/FTP storage (as required).
- Optionally capture any other important information.
- For switches, run `show ip interface brief`.
- For routers, run `show ip routing`, which is the routing table.

Phase 3: Reloading and Post-upgrade Check

10 During device reload (reloading state):

- Check if the device is on the network .

11 After the reload (the device is back on the network):

- Log back into the device and perform a post-upgrade verification using before and after configurations.
- Optionally, send an email notification to the engineer at the end of the upgrade completion.

We have reviewed the manual IOS upgrade processes shown previously, but how can we translate each item into Python code? Where do we begin, and is it even possible to create such sophisticated IOS upgrading applications in production? Compared to what other tools offer us in the current market, there are many tasks (and an engineer's logical thinking) that must be translated into code lines (a program). Next, let's study Figure [17-15](#), which is my translation of Table [17-2](#) and Figure [17-14](#) into automatable tasks.

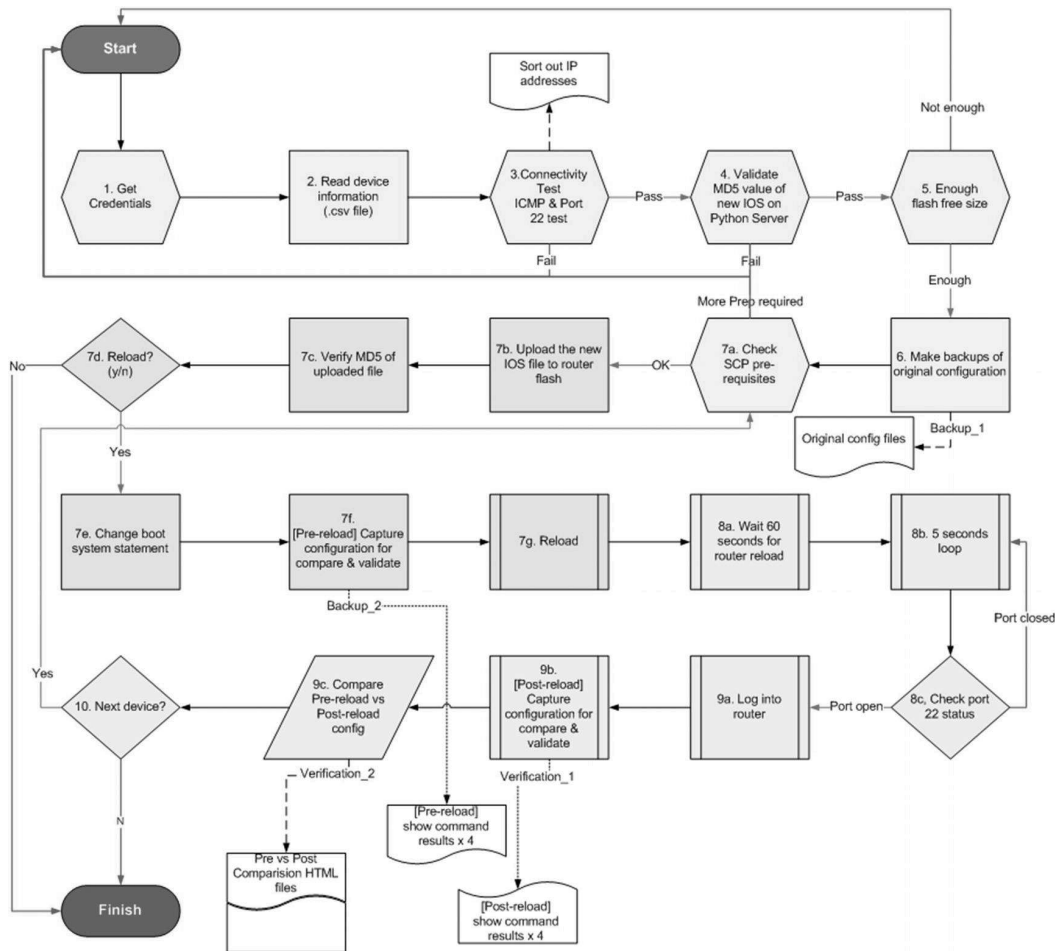


Figure 17-15. Cisco IOS/IOS XE automated upgrade, suggested workflow example

Figure [17-15](#) is almost a direct translation of the tasks discussed in Table [17-2](#), and the numbers correspond to that of Table [17-2](#). You should carefully study it before moving on to the next chapter. In Chapter [18](#), we will turn each step in the upgrade process into smaller snippets of Python code. In Chapter [18](#), tools developed for each process will become individual tools themselves. Then, in Chapter [19](#), you will combine them into a single tool (application) to replace the Cisco IOS upgrade tasks from end to end, that is, the pre-check, IOS upgrade, and post-check processes.

Summary

The processes discussed in this chapter are still somewhat generalized, but if you want to be more detailed, you can break down each task into even smaller tasks. A programmer without essential networking background will have little or no idea of the processes typically performed by experienced network engineers. Only network engineers who can write code will string together each task into sequenced tasks and cover all the

essential tasks required to turn their actions into actionable code lines. So, in this vein, the foreseeable future for network (including security) engineers is bright as they cannot be replaced by pure application developers or untrained artificial intelligence (AI). Still, organizations will prefer engineers with coding skills over traditional engineers with no coding skills. Perhaps most network engineers will agree that, soon, AI will be replacing much of the work we perform today. At least this is true for the initial design and development of these AI platforms, because they must be initiated by experienced and intelligent network engineers, not the application developers.

Answers collapsed